

✓

GenAI Assignment

Name: Yesha Pandya

Enrolment Number: 23BT04175

Div: A

Category: Fast Learner

✓

Q1. VAE with a 2-D Latent Space

Scenario:

A team wants to see how a VAE arranges digit images in a 2-D latent space and to generate new digits from it.

Task:

- Implement a VAE on MNIST with a 2-D latent space (loss = reconstruction + KL) and train for 10 epochs.
- Plot the 2-D latent points coloured by digit label and generate 10 new digits by sampling $z \sim N(0, I)$.
- Explain in 4 lines the role of the reconstruction loss and the KL term.

```
# Import libraries
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np

# Set random seed for reproducibility
torch.manual_seed(42)

# Set hyperparameters
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
batch_size = 128
latent_dim = 2
epochs = 10
lr = 1e-3
```

✓

Load + Check Dataset

```
# Load dataset
transform = transforms.ToTensor()
train_dataset = datasets.MNIST(root='./data', train=True, transform=transform, download=True)
test_dataset = datasets.MNIST(root='./data', train=False, transform=transform, download=True)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

# Sample check
sample_imgs, sample_labels = next(iter(train_loader))
print(f"Number of train samples: {len(train_dataset)}")
print(f"Number of test samples: {len(test_dataset)}")
print(f"Image batch shape: {sample_imgs.shape}")

# Visualize first 5 training samples
print("\n Sample Images from MNIST dataset:")
fig, axes = plt.subplots(1, 5, figsize=(10, 2))
for i in range(5):
    axes[i].imshow(sample_imgs[i].squeeze(), cmap='gray')
    axes[i].set_title(f"Label: {sample_labels[i].item()}")
    axes[i].axis('off')
plt.show()
```

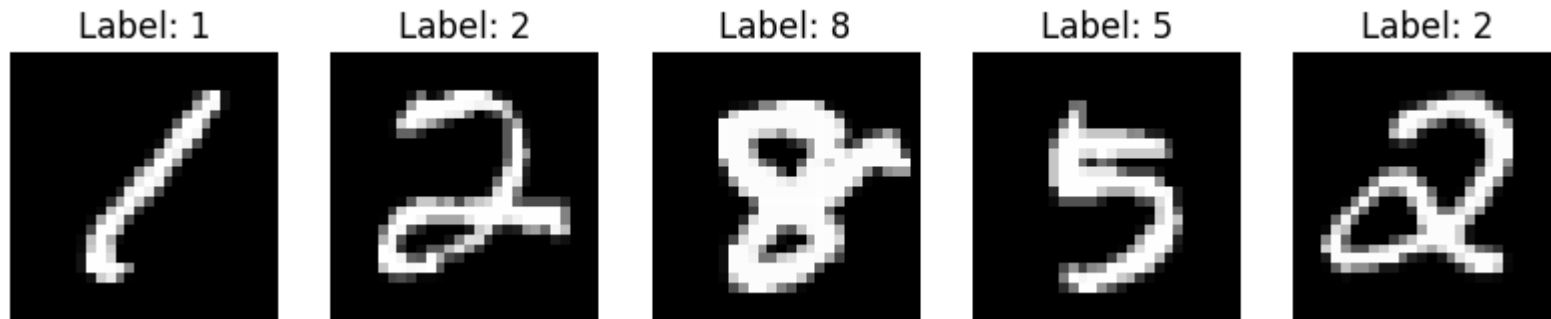
```
100%|██████████| 9.91M/9.91M [00:00<00:00, 14.0MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 352kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 3.22MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 4.43MB/s]
```

Number of train samples: 60000

Number of test samples: 10000

Image batch shape: torch.Size([128, 1, 28, 28])

Sample Images from MNIST dataset:



✓ VAE Architecture

```
# Define VAE architecture
class ConvVAE(nn.Module):
    def __init__(self, latent_dim=2):
        super(ConvVAE, self).__init__()

        # Encoder (we'll use 2D Convolutions to preserve image geometry)
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, stride=2, padding=1), # Output: 32 x 14 x 14
            nn.BatchNorm2d(32),
            nn.LeakyReLU(0.2),
            nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1), # Output: 64 x 7 x 7
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.2),
            nn.Flatten(), # 64*7*7 = 3136 (flattened latent vector)
            nn.Linear(3136, 128),
            nn.LeakyReLU(0.2)
        )

        # latent space heads (2D)
        self.fc_mu = nn.Linear(128, latent_dim)
        self.fc_logvar = nn.Linear(128, latent_dim)

        # decoder projection
        self.decoder_input = nn.Linear(latent_dim, 128)
        self.decoder_fc = nn.Sequential(
            nn.Linear(128, 3136),
            nn.LeakyReLU(0.2)
        )

        # Decoder (transposed convolutions for crisp spatial reconstruction)
        self.decoder_conv = nn.Sequential(
            nn.ConvTranspose2d(64, 32, kernel_size=3, stride=2, padding=1, output_padding=1), # Output: 32 x
            nn.BatchNorm2d(32),
            nn.LeakyReLU(0.2),
            nn.ConvTranspose2d(32, 1, kernel_size=3, stride=2, padding=1, output_padding=1), # Output: 1 x
            nn.Sigmoid() # pixel range = [0, 1]
        )

    def encode(self, x):
        h = self.encoder(x)
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        # reshape 1D latent vector back into 3D feature map before deconvolution
        h = self.decoder_input(z)
        h = self.decoder_fc(h)
        h = h.view(-1, 64, 7, 7)
        return self.decoder_conv(h)

    def forward(self, x):
        mu, logvar = self.encode(x)
```

```
z = self.reparameterize(mu, logvar)
return self.decode(z), mu, logvar

# Instantiate ConvVAE model
model = ConvVAE(latent_dim=latent_dim).to(device)
print(model)

# Parameter Counter
total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"\nConvVAE Initialized with {total_params:,} trainable parameters.")

ConvVAE(
  (encoder): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): LeakyReLU(negative_slope=0.2)
    (3): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): LeakyReLU(negative_slope=0.2)
    (6): Flatten(start_dim=1, end_dim=-1)
    (7): Linear(in_features=3136, out_features=128, bias=True)
    (8): LeakyReLU(negative_slope=0.2)
  )
  (fc_mu): Linear(in_features=128, out_features=2, bias=True)
  (fc_logvar): Linear(in_features=128, out_features=2, bias=True)
  (decoder_input): Linear(in_features=2, out_features=128, bias=True)
  (decoder_fc): Sequential(
    (0): Linear(in_features=128, out_features=3136, bias=True)
    (1): LeakyReLU(negative_slope=0.2)
  )
  (decoder_conv): Sequential(
    (0): ConvTranspose2d(64, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): LeakyReLU(negative_slope=0.2)
    (3): ConvTranspose2d(32, 1, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (4): Sigmoid()
  )
)

ConvVAE Initialized with 844,805 trainable parameters.
```

✓ **Loss functions + Training loop**

```
def vae_loss_function(recon_x, x, mu, logvar):
    # BCE over 28x28 spatial dimensions
    BCE = nn.functional.binary_cross_entropy(recon_x, x, reduction='sum')
    KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return BCE + KLD, BCE, KLD

optimizer = optim.Adam(model.parameters(), lr=lr)

print("Start ConvVAE Training Loop ..")
model.train()
for epoch in range(1, epochs + 1):
    total_train_loss, total_bce, total_kld = 0, 0, 0
    for x, _ in train_loader:
        x = x.to(device) # x is kept as 4D tensor: [batch_size, 1, 28, 28]
        optimizer.zero_grad()

        # Pass full 4D image tensor directly to ConvVAE
        recon_batch, mu, logvar = model(x)

        loss, bce, kld = vae_loss_function(recon_batch, x, mu, logvar)
        loss.backward()
        optimizer.step()

        total_train_loss += loss.item()
        total_bce += bce.item()
        total_kld += kld.item()

    avg_loss = total_train_loss / len(train_loader.dataset)
    avg_bce = total_bce / len(train_loader.dataset)
    avg_kld = total_kld / len(train_loader.dataset)
    print(f"Epoch {epoch:02d}/{epochs:02d} | Total Loss: {avg_loss:.2f} | BCE: {avg_bce:.2f} | KLD: {avg_kld:.2f}")

Start ConvVAE Training Loop ..
Epoch 01/10 | Total Loss: 206.14 | BCE: 201.78 | KLD: 4.36
Epoch 02/10 | Total Loss: 161.81 | BCE: 156.74 | KLD: 5.07
Epoch 03/10 | Total Loss: 157.82 | BCE: 152.47 | KLD: 5.35
Epoch 04/10 | Total Loss: 155.75 | BCE: 150.26 | KLD: 5.50
Epoch 05/10 | Total Loss: 154.27 | BCE: 148.65 | KLD: 5.62
```

Epoch 06/10		Total Loss: 153.35		BCE: 147.67		KLD: 5.68
Epoch 07/10		Total Loss: 152.52		BCE: 146.79		KLD: 5.74
Epoch 08/10		Total Loss: 151.82		BCE: 146.03		KLD: 5.79
Epoch 09/10		Total Loss: 151.33		BCE: 145.51		KLD: 5.82
Epoch 10/10		Total Loss: 150.89		BCE: 145.02		KLD: 5.86

Map high-dimensional input into latent space + visualize

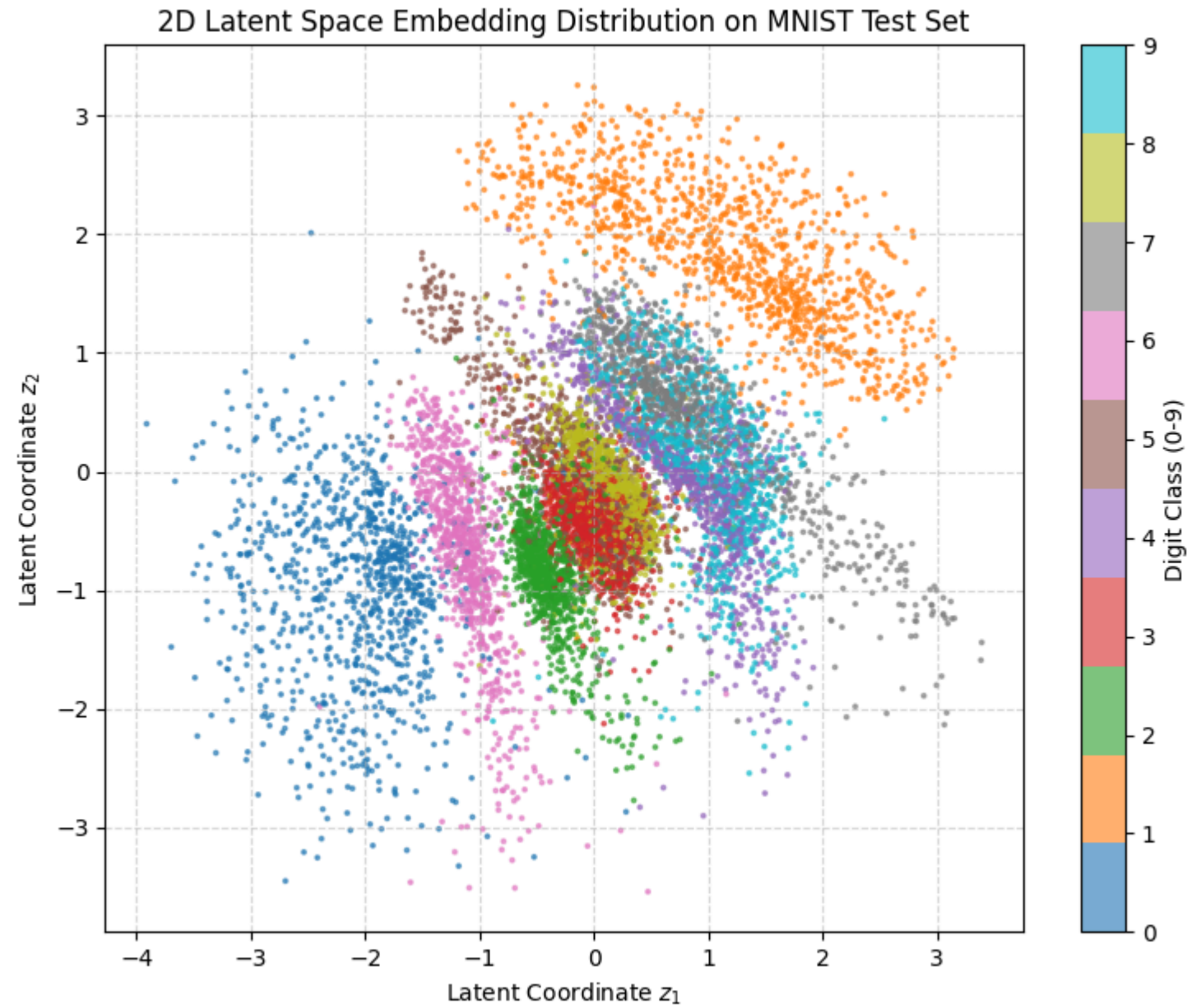
```
model.eval() # switch VAE model to evaluation mode (disables batchnorm/dropout updates)
latent_points, labels_list = [], [] # containers to store 2D latent coordinates and corresponding digit labels

# Disable gradient calculations since we are doing inference/visualization only
with torch.no_grad():
    for x, y in test_loader:
        x = x.to(device)
        # flatten the 28x28 image into a 784 vector and pass through VAE Encoder
        # encoder returns mean (mu) and log variance (logvar) of the 2D distribution
        mu, _ = model.encode(x)
        # extract 'mu' as the deterministic 2D location (z1, z2) for each digit image
        latent_points.append(mu.cpu().numpy())
        labels_list.append(y.numpy())

# Concatenate all mini-batch arrays into single 2D NumPy arrays
latent_points = np.concatenate(latent_points, axis=0)
labels_list = np.concatenate(labels_list, axis=0)
```

```
# Visualize the 2D scatter plot of latent embedding space
plt.figure(figsize=(9, 7))
scatter = plt.scatter(latent_points[:, 0], latent_points[:, 1], c=labels_list, cmap='tab10', alpha=0.6, s=3)
plt.colorbar(scatter, label='Digit Class (0-9)')
plt.xlabel("Latent Coordinate $z_1$")
plt.ylabel("Latent Coordinate $z_2$")
plt.title("2D Latent Space Embedding Distribution on MNIST Test Set")
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()

print(f"Embedded {len(latent_points)} points into 2D space.")
print(f"z1 range: [{latent_points[:,0].min():.2f}, {latent_points[:,0].max():.2f}]")
print(f"z2 range: [{latent_points[:,1].min():.2f}, {latent_points[:,1].max():.2f}]")
```



Embedded 10000 points into 2D space.
z1 range: [-3.91, 3.39]
z2 range: [-3.54, 3.26]

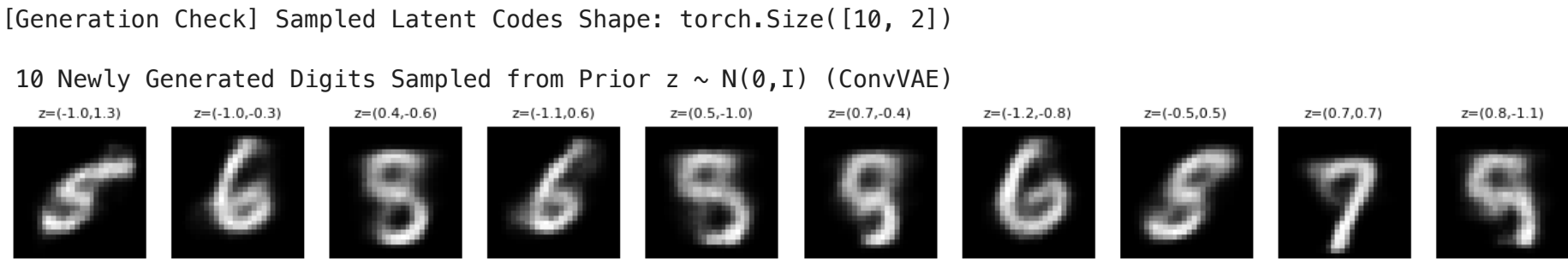
Use Prior Sampling to generate brand new digit images

```
model.eval()
with torch.no_grad():
    # Sample 10 random 2D points from standard normal distribution
    z_samples = torch.randn(10, latent_dim).to(device)

    # Decoder generates crisp (1, 28, 28) images directly via deconvolution
    generated_imgs = model.decode(z_samples).cpu().squeeze(1)

print(f"[Generation Check] Sampled Latent Codes Shape: {z_samples.shape}")

print("\n 10 Newly Generated Digits Sampled from Prior z ~ N(0,I) (ConvVAE)")
fig, axes = plt.subplots(1, 10, figsize=(16, 2))
for i in range(10):
    axes[i].imshow(generated_imgs[i], cmap='gray')
    axes[i].set_title(f"z=({z_samples[i,0]:.1f},{z_samples[i,1]:.1f})", fontsize=8)
    axes[i].axis('off')
plt.show()
```



Role of Reconstruction Loss and KL Divergence Term

- 1. **Reconstruction Loss** (e.g., Binary Cross-Entropy or Mean Squared Error)
 - It measures how accurately the decoder reconstructs the original input image from the latent vector z .
 - It forces the autoencoder to retain fine structural details and fidelity of the input image in its output.
- 2. **KL Divergence Loss**
 - It acts as a regularizer by penalizing the divergence between the learned latent distribution $q(z|x)$ and a standard Gaussian prior $p(z) = N(0, I)$.
 - It enforces continuity and completeness in the latent space. This ensures smooth interpolation and allows valid generation by sampling from $N(0, I)$.

Q2. DCGAN on Fashion-MNIST

Scenario:

A game studio needs a generator that creates small clothing images automatically.

Task:

- Implement a DCGAN on Fashion-MNIST:
 - Generator with ConvTranspose2d + BatchNorm + ReLU (Tanh output)
 - Discriminator with strided Conv2d + LeakyReLU(0.2).
- Train for 15 epochs with Adam ($\text{lr} = 2\text{e-}4$, $\beta_1 = 0.5$). Save an 8x8 image grid at epoch 1 and epoch 15 using the same fixed noise.
- Plot both loss curves and write 3 lines on what you observe (image quality, any instability).

Hint: Follow the official PyTorch DCGAN tutorial and adapt it to 28x28 grayscale images.

```
# Import libraries
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import torchvision.utils as vutils
import matplotlib.pyplot as plt
import numpy as np
```

```
# Set reproducibility seed
torch.manual_seed(42)

# Hardware setup + hyperparameters
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
batch_size = 128
nz = 100 # Z latent noise vector
epochs = 15
lr = 2e-4
beta1 = 0.5 # Beta1 hyperparameter for Adam optimizer
```

Load + check dataset

```
# Image normalization to [-1, 1] for Tanh output alignment
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)) # adjust scale [0,1] -> [-1,1]
])

# Load Fashion-MNIST dataset
train_dataset = datasets.FashionMNIST(root='./data', train=True, transform=transform, download=True)
dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, drop_last=True)

# Fetch a single batch of real images to verify setup
real_batch, real_labels = next(iter(dataloader))

print(f"Fashion-MNIST Training Samples: {len(train_dataset)}")
print(f"Image Tensor Shape: {real_batch.shape} (Range: [{real_batch.min():.1f}, {real_batch.max():.1f}])")

# Visualize a sample 8x8 grid of REAL Fashion-MNIST clothing items
real_grid = vutils.make_grid(real_batch[:64], padding=2, normalize=True, nrow=8)

plt.figure(figsize=(6, 6))
plt.imshow(np.transpose(real_grid.cpu().numpy(), (1, 2, 0)))
print("\n Sample Real Fashion-MNIST Assets (8x8 Grid)")
plt.axis("off")
plt.show()
```

```
100%|██████████| 26.4M/26.4M [00:02<00:00, 12.8MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 202kB/s]
100%|██████████| 4.42M/4.42M [00:01<00:00, 3.76MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 24.2MB/s]
Fashion-MNIST Training Samples: 60000
Image Tensor Shape: torch.Size([128, 1, 28, 28]) (Range: [-1.0, 1.0])
```

Sample Real Fashion-MNIST Assets (8x8 Grid)



Model architecture + weight initialization

```

# Custom weight initialization function recommended by the DCGAN paper
def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)

# Generator (ConvTranspose2d → BatchNorm → ReLU → Tanh)
class Generator(nn.Module):
    def __init__(self):
        super(Generator, self).__init__()
        self.main = nn.Sequential(
            # Input: latent noise z (100 x 1 x 1) → Output: 256 x 7 x 7
            nn.ConvTranspose2d(nz, 256, kernel_size=7, stride=1, padding=0, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(True),

            # Input: 256 x 7 x 7 → Output: 128 x 14 x 14
            nn.ConvTranspose2d(256, 128, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(True),

            # Input: 128 x 14 x 14 → Output: 1 x 28 x 28
            nn.ConvTranspose2d(128, 1, kernel_size=4, stride=2, padding=1, bias=False),
            nn.Tanh() # Produces pixel values bounded in [-1, 1]
        )

    def forward(self, input):
        return self.main(input)

# Discriminator (Strided Conv2d → LeakyReLU(0.2) → Sigmoid)
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            # Input: 1 x 28 x 28 → Output: 128 x 14 x 14
            nn.Conv2d(1, 128, kernel_size=4, stride=2, padding=1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),

            # Input: 128 x 14 x 14 → Output: 256 x 7 x 7
            nn.Conv2d(128, 256, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2, inplace=True),

            # Input: 256 x 7 x 7 → Output: 1 x 1 x 1
            nn.Conv2d(256, 1, kernel_size=7, stride=1, padding=0, bias=False),
            nn.Sigmoid()
        )

    def forward(self, input):
        return self.main(input)

# Instantiate models
netG = Generator().to(device)
netD = Discriminator().to(device)

# Apply custom weights initialization
netG.apply(weights_init)
netD.apply(weights_init)

# Verify tensor dimensions with dummy forward passes
dummy_z = torch.randn(1, nz, 1, 1, device=device)
dummy_img = netG(dummy_z)
dummy_pred = netD(dummy_img)

print(f"[Generator] Dummy Latent Vector {dummy_z.shape} → Synthetic Image Output: {dummy_img.shape}")
print(f"[Discriminator] Synthetic Image Input {dummy_img.shape} → Validity Score Shape: {dummy_pred.shape}")

[Generator] Dummy Latent Vector torch.Size([1, 100, 1, 1]) → Synthetic Image Output: torch.Size([1, 1, 28, 28])
[Discriminator] Synthetic Image Input torch.Size([1, 1, 28, 28]) → Validity Score Shape: torch.Size([1, 1, 1, 1])

```

✓ DCGAN training loop

```

# Binary Cross Entropy Loss function
criterion = nn.BCELoss()

```

```

# Fixed noise tensor (64 vectors) used to track exact same latent coordinates across Epoch 1 and Epoch 15
fixed_noise = torch.randn(64, nz, 1, 1, device=device)

# Setup Adam optimizers with beta1 = 0.5
optimizerD = optim.Adam(netD.parameters(), lr=lr, betas=(beta1, 0.999))
optimizerG = optim.Adam(netG.parameters(), lr=lr, betas=(beta1, 0.999))

G_losses = []
D_losses = []

print("Starting DCGAN Training Loop ..")

for epoch in range(1, epochs + 1):
    for i, (real_imgs, _) in enumerate(dataloader):

        # Step 1: Update Discriminator → maximize log(D(x)) + log(1 - D(G(z)))
        netD.zero_grad()
        real_imgs = real_imgs.to(device)
        b_size = real_imgs.size(0)

        # Real and fake binary targets
        label_real = torch.full((b_size,), 1.0, device=device)
        label_fake = torch.full((b_size,), 0.0, device=device)

        # Forward pass real batch through D
        output_real = netD(real_imgs).view(-1)
        errD_real = criterion(output_real, label_real)
        errD_real.backward()

        # Generate fake batch and forward pass through D
        noise = torch.randn(b_size, nz, 1, 1, device=device)
        fake_imgs = netG(noise)
        output_fake = netD(fake_imgs.detach()).view(-1)
        errD_fake = criterion(output_fake, label_fake)
        errD_fake.backward()

        errD = errD_real + errD_fake
        optimizerD.step()

        # Step 2: Update Generator → maximize log(D(G(z)))
        netG.zero_grad()
        # Generator wants Discriminator to believe fake images are Real (1.0)
        output_g = netD(fake_imgs).view(-1)
        errG = criterion(output_g, label_real)
        errG.backward()
        optimizerG.step()

        # Record losses for visualization
        G_losses.append(errG.item())
        D_losses.append(errD.item())

    # Logging progress per epoch
    print(f"Epoch [{epoch:02d}/{epochs:02d}] | Loss_D: {errD.item():.4f} | Loss_G: {errG.item():.4f}")

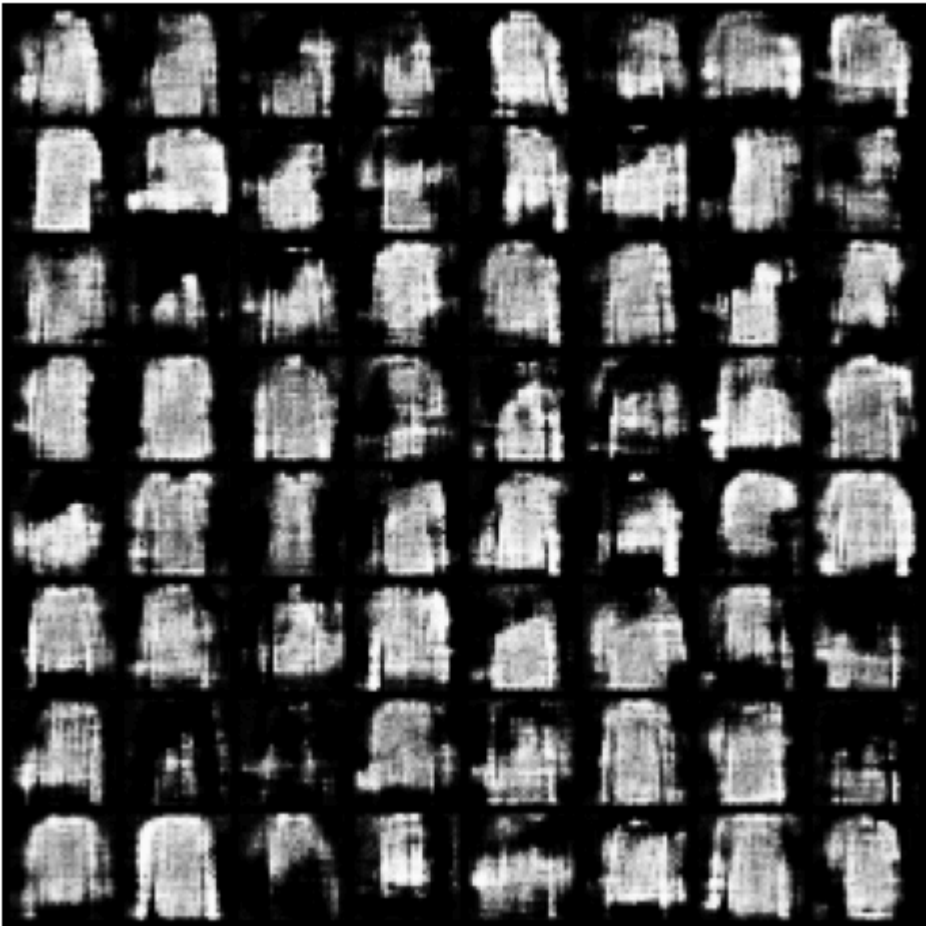
    # Capture + display 8x8 grid at epoch 01 and epoch 15
    if epoch == 1 or epoch == 15:
        netG.eval()
        with torch.no_grad():
            fake_samples = netG(fixed_noise).detach().cpu()
            grid = vutils.make_grid(fake_samples, padding=2, normalize=True, nrow=8)

            plt.figure(figsize=(6, 6))
            plt.imshow(np.transpose(grid.numpy(), (1, 2, 0)))
            plt.title(f"DCGAN Generated 8x8 Grid - Epoch {epoch}", fontsize=12)
            plt.axis("off")
            plt.show()
        netG.train()

```

Starting DCGAN Training Loop ..
Epoch [01/15] | Loss_D: 0.1910 | Loss_G: 3.1965

DCGAN Generated 8x8 Grid - Epoch 1



Epoch [02/15]	Loss_D: 0.6182	Loss_G: 1.7356
Epoch [03/15]	Loss_D: 0.6783	Loss_G: 1.3870
Epoch [04/15]	Loss_D: 0.8076	Loss_G: 1.6760
Epoch [05/15]	Loss_D: 0.6914	Loss_G: 1.8331
Epoch [06/15]	Loss_D: 1.0900	Loss_G: 3.2801
Epoch [07/15]	Loss_D: 0.7620	Loss_G: 1.4332
Epoch [08/15]	Loss_D: 0.9719	Loss_G: 1.8204
Epoch [09/15]	Loss_D: 0.8605	Loss_G: 1.3590
Epoch [10/15]	Loss_D: 0.9330	Loss_G: 1.7802
Epoch [11/15]	Loss_D: 0.7982	Loss_G: 1.6425
Epoch [12/15]	Loss_D: 1.0667	Loss_G: 1.1719
Epoch [13/15]	Loss_D: 0.9174	Loss_G: 1.9189
Epoch [14/15]	Loss_D: 0.9347	Loss_G: 1.9476
Epoch [15/15]	Loss_D: 0.9261	Loss_G: 1.3090

DCGAN Generated 8x8 Grid - Epoch 15



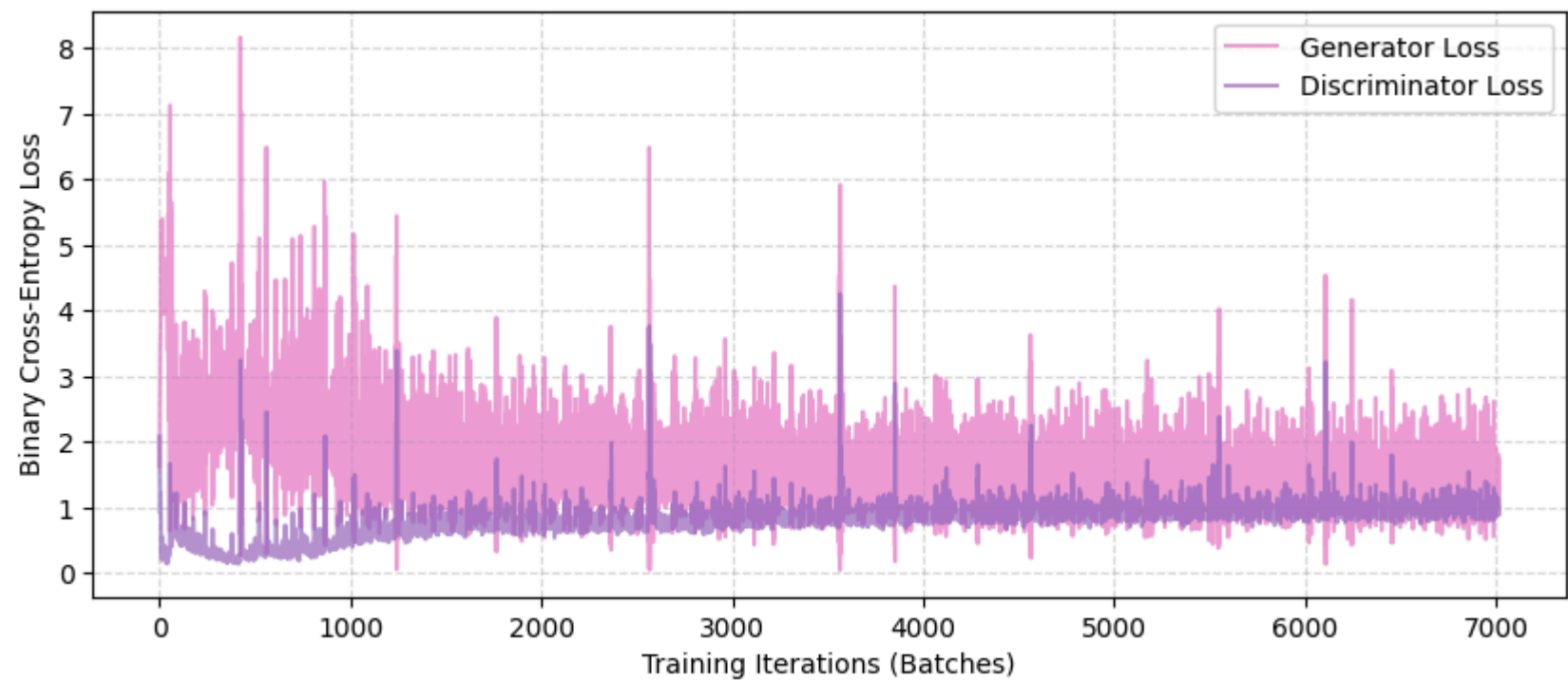
Adversial loss curves visualization

```
print("DCGAN Generator and Discriminator Loss Dynamics (Fashion-MNIST)")
plt.figure(figsize=(10, 4))
plt.plot(G_losses, label="Generator Loss", alpha=0.7, color='tab:pink')
plt.plot(D_losses, label="Discriminator Loss", alpha=0.7, color='tab:purple')
plt.xlabel("Training Iterations (Batches)")
plt.ylabel("Binary Cross-Entropy Loss")
```

```
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()

print(f"[Training Complete] Executed {len(G_losses)} total batch update iterations.")
```

DCGAN Generator and Discriminator Loss Dynamics (Fashion-MNIST)



[Training Complete] Executed 7020 total batch update iterations.

Observations

- 1. **Image Quality Progression:**
 - At Epoch 1 → generated samples display noisy, unformed gray blobs
 - By Epoch 15 → the Generator synthesizes recognizable clothing silhouettes (shirts/trousers/boots/bags) with sharp borders.
- 2. **Training Stability:** The loss curves oscillate smoothly without exploding or dropping to zero ⇒ shows that setting Adam $\beta_1 = 0.5$ and learning rate 2×10^{-4} successfully maintained dynamic equilibrium between G and D .
- 3. **Absence of Mode Collapse:** The 8×8 grid at Epoch 15 displays diverse clothing categories across all 64 slots ⇒ confirms that batch normalization and strided convolutions prevented mode collapse.

Q3. Conditional GAN: Generate on Demand

Scenario:

A digit app must let the user choose which digit (0–9) to generate.

Task:

- Implement a CGAN on MNIST that adds the label to both the Generator input and the Discriminator input.
- Generate one row of 10 images, one for each digit 0–9.
- Explain in 4 lines how the label is given to the Generator and the Discriminator and how a CGAN differs from a normal GAN.

Hint: Use an embedding layer for the label and concatenate it with z (Generator) or with the image (Discriminator)

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Hyperparameters
batch_size = 128
z_dim = 100          # Size of noise vector
num_classes = 10     # Digits 0 - 9
emb_dim = 50         # Size of the label embedding vector
epochs = 12
lr = 2e-4
```

```
# MNIST Dataset (Normalized to [-1,1] for Tanh)
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

dataset = datasets.MNIST(root='./data', train=True, transform=transform, download=True)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True, drop_last=True)

print(f"Dataset Loaded: {len(dataset)} MNIST images.")

Dataset Loaded: 60000 MNIST images.
```

✓ CGAN architecture (Generator + Discriminator)

```
# CONDITIONAL GENERATOR
class ConditionalGenerator(nn.Module):
    def __init__(self):
        super(ConditionalGenerator, self).__init__()
        # Embedding layer for the label
        self.label_emb = nn.Embedding(num_classes, emb_dim)

        # Dense network taking concatenated [z, label] as input
        self.net = nn.Sequential(
            nn.Linear(z_dim + emb_dim, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 512),
            nn.BatchNorm1d(512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 1024),
            nn.BatchNorm1d(1024),
            nn.LeakyReLU(0.2),
            nn.Linear(1024, 784),
            nn.Tanh() # Output range [-1, 1]
        )

    def forward(self, z, labels):
        # 1. Embed the label
        c = self.label_emb(labels)
        # 2. Concatenate noise z and embedded label c
        x = torch.cat([z, c], dim=1)
        # 3. Generate and reshape to image dimensions
        return self.net(x).view(-1, 1, 28, 28)

# CONDITIONAL DISCRIMINATOR
class ConditionalDiscriminator(nn.Module):
    def __init__(self):
        super(ConditionalDiscriminator, self).__init__()
        self.label_emb = nn.Embedding(num_classes, emb_dim)

        # Dense network taking concatenated [image_pixels, label] as input
        self.net = nn.Sequential(
            nn.Linear(784 + emb_dim, 512),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),
            nn.Linear(256, 1),
            nn.Sigmoid()
        )

    def forward(self, imgs, labels):
        # 1. Embed the label
        c = self.label_emb(labels)
        # 2. Flatten image and concatenate with embedded label
        x = torch.cat([imgs.view(imgs.size(0), -1), c], dim=1)
        # 3. Output probability
        return self.net(x)

c_netG = ConditionalGenerator().to(device)
c_netD = ConditionalDiscriminator().to(device)

print(f"Conditional Generator input size: z_dim ({z_dim}) + emb_dim ({emb_dim}) = {z_dim + emb_dim}")
print(f"Conditional Discriminator input size: image ({28*28}) + emb_dim ({emb_dim}) = {784 + emb_dim}")

Conditional Generator input size: z_dim (100) + emb_dim (50) = 150
Conditional Discriminator input size: image (784) + emb_dim (50) = 834
```

CGAN adversarial training loop

```

criterion = nn.BCELoss()
optG = optim.Adam(c_netG.parameters(), lr=lr, betas=(0.5, 0.999))
optD = optim.Adam(c_netD.parameters(), lr=lr, betas=(0.5, 0.999))

print("Starting CGAN Training Loop..")

for epoch in range(1, epochs + 1):
    for i, (imgs, labels) in enumerate(dataloader):
        b_size = imgs.size(0)
        imgs, labels = imgs.to(device), labels.to(device)

        # Targets
        real_target = torch.ones(b_size, 1, device=device)
        fake_target = torch.zeros(b_size, 1, device=device)

        # Train Discriminator
        optD.zero_grad()
        # D evaluates Real Images + Real Labels
        out_real = c_netD(imgs, labels)
        loss_real = criterion(out_real, real_target)

        # D evaluates Fake Images + Fake Labels
        z = torch.randn(b_size, z_dim, device=device)
        fake_imgs = c_netG(z, labels)
        out_fake = c_netD(fake_imgs.detach(), labels)
        loss_fake = criterion(out_fake, fake_target)

        loss_D = loss_real + loss_fake
        loss_D.backward()
        optD.step()

        # Train Generator
        optG.zero_grad()
        # G wants D to classify Fake Images + Real Labels as Real
        out_fake_G = c_netD(fake_imgs, labels)
        loss_G = criterion(out_fake_G, real_target)

        loss_G.backward()
        optG.step()

    print(f"Epoch [{epoch:02d}/{epochs:02d}] | Loss_D: {loss_D.item():.4f} | Loss_G: {loss_G.item():.4f}")

```

```

Starting CGAN Training Loop..
Epoch [01/12] | Loss_D: 1.1114 | Loss_G: 0.9429
Epoch [02/12] | Loss_D: 1.0439 | Loss_G: 1.7003
Epoch [03/12] | Loss_D: 1.0075 | Loss_G: 1.3581
Epoch [04/12] | Loss_D: 0.8215 | Loss_G: 2.1932
Epoch [05/12] | Loss_D: 0.8840 | Loss_G: 1.5163
Epoch [06/12] | Loss_D: 0.8608 | Loss_G: 2.5036
Epoch [07/12] | Loss_D: 0.7181 | Loss_G: 1.5410
Epoch [08/12] | Loss_D: 0.6259 | Loss_G: 2.5648
Epoch [09/12] | Loss_D: 0.8137 | Loss_G: 2.1486
Epoch [10/12] | Loss_D: 0.6433 | Loss_G: 1.7799
Epoch [11/12] | Loss_D: 0.7903 | Loss_G: 2.0093
Epoch [12/12] | Loss_D: 0.5608 | Loss_G: 2.0182

```

Targeted generation on digits 0-9

```

c_netG.eval()

# Create a tensor of exact labels we want: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
target_labels = torch.arange(0, 10, dtype=torch.long, device=device)

# Sample one random noise vector per target label
fixed_z = torch.randn(10, z_dim, device=device)

with torch.no_grad():
    # Pass targeted labels alongside noise to force specific outputs
    generated_row = c_netG(fixed_z, target_labels).cpu()

# Plotting the sequence
fig, axes = plt.subplots(1, 10, figsize=(15, 2))
for i in range(10):
    # Denormalize from [-1, 1] back to [0,1] for visualization
    img = (generated_row[i].squeeze() + 1) / 2.0

```

```
axes[i].imshow(img, cmap='gray')
axes[i].set_title(f"Label: {target_labels[i].item()}", fontsize=12, fontweight='bold')
axes[i].axis('off')

print("CGAN Output: Targeted Generation on Demand")
plt.show()
```



Q4 - Theory is handwritten - attached PDF after Q5

✓ **Q5. Introduction to Diffusion Models**

Scenario:

A team is learning about Diffusion Models, the technology behind many modern image generators.

Task:

- Explain in 5 lines, with a simple diagram, the forward process (adding noise step by step) and the reverse process (removing noise to create an image).
- Practical: load one image and add Gaussian noise using
$$x_{noisy} = x + \sigma \cdot \epsilon(\epsilon \sim N(0, 1))$$
for $\sigma = 0.1, 0.5$ and 1.0 . Display the three noisy images.
- Write a short table choosing GAN, VAE or Diffusion for: realistic face generation, molecule design and image style transfer, with one line of reasoning each.

Hint: `np.random.randn(*image.shape)` creates ϵ with the same shape as the image Submit: Explanation with diagram, code output and the table.

✓ **Load image + add Gaussian noise**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Set your uploaded image filename here (works for both COLOR and GRAYSCALE)
IMAGE_NAME = 'aelin.jpg'

# 1. Read Image via OpenCV
img_bgr = cv2.imread(IMAGE_NAME)

if img_bgr is None:
    raise FileNotFoundError(f"Could not find '{IMAGE_NAME}'. Make sure it's uploaded to the Colab sidebar!")

# Detect color vs grayscale and normalize pixel values to [0, 1]
if len(img_bgr.shape) == 3:
    img = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB) / 255.0
    is_color = True
else:
    img = img_bgr / 255.0
    is_color = False

# Display uploaded image
plt.figure(figsize=(4, 4))
plt.imshow(img, cmap='gray' if not is_color else None)
plt.title("Loaded Original Image ($x_0$)", fontweight='bold')
plt.axis('off')
plt.show()
```

Loaded Original Image (x_0)



```
# Progressive Noise Addition ( $\sigma = 0.1, 0.5, 1.0$ )
sigmas = [0.1, 0.5, 1.0]
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for i, sigma in enumerate(sigmas):
    # Gaussian noise formula:  $x_{\text{noisy}} = x + \sigma \cdot \epsilon$ , where  $\epsilon \sim N(0, 1)$ 
    epsilon = np.random.randn(*img.shape)
    img_noisy = img + sigma * epsilon

    # Clip pixel values back to valid range [0, 1] for display
    img_noisy_clipped = np.clip(img_noisy, 0.0, 1.0)

    axes[i].imshow(img_noisy_clipped, cmap='gray' if not is_color else None)
    axes[i].set_title(f"Noisy Stage {i+1} ( $\sigma = \{sigma\}$ )", fontweight='bold')
    axes[i].axis('off')

print("Forward Diffusion Process: Step-by-Step Noise Addition")
plt.tight_layout()
plt.show()
```

Forward Diffusion Process: Step-by-Step Noise Addition

Noisy Stage 1 ($\sigma = 0.1$)



Noisy Stage 2 ($\sigma = 0.5$)



Noisy Stage 3 ($\sigma = 1.0$)



(Flowchart and theory for Q4 and Q5 are handwritten - attached below)

GenAI CEC Assignment.

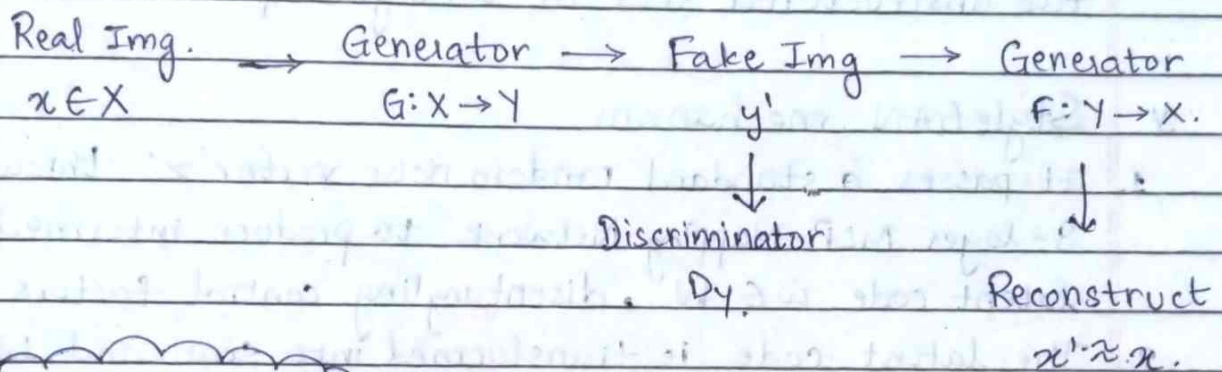
Date _____
Page _____

23BT04175.

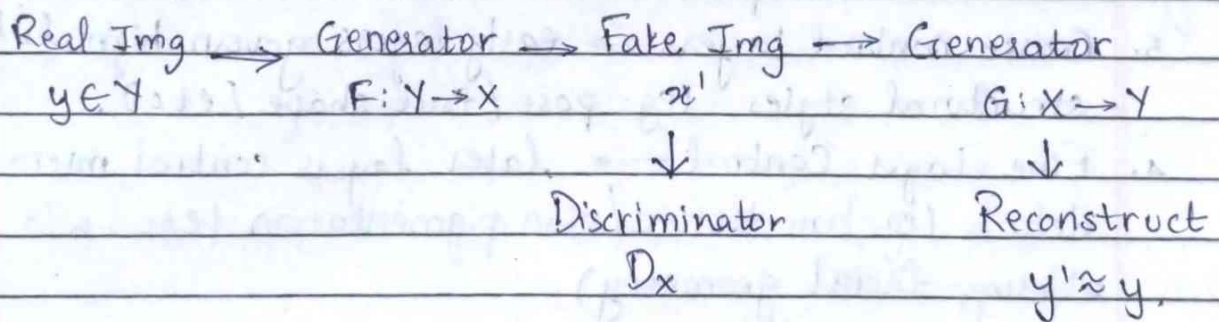
Q4. CycleGAN and StyleGAN.

* CycleGAN Block Diagram.

1. Forward Cycle (Source Domain X to Target Domain Y)



2. Backward Cycle (Source Domain Y to Target Domain X)



* Cycle-consistency in CycleGAN.

↳ Enforces that an img translated from Domain X to Domain Y must be reconstructible back to Domain X.

↳ This bidirectional loss prevents the generator from discarding input spatial features. This ensures that pose and background are preserved while changing domain attributes (eg: adding stripes for a zebra).

* Unpaired Translation in CycleGAN.

Traditional img translation relies on pixel-aligned pairs (x_i, y_i) (eg: photo to sketch) evaluated using $L1/L2$ loss.

CycleGAN pairs adversarial distribution matching (D_x, D_y) with self-supervised cycle loss, allowing translation b/w two unstructured sets of unaligned photos.

* StyleGAN mechanism

1. It passes a standard random noise vector 'z' through a 8-layer MLP Mapping Network to produce intermediate latent code $w \in W$, disentangling control factors.
2. The latent code is transformed into scale and bias style parameters (γ, β) & injected into each convolutional block via Adaptive Instance Normalization (AdaIN).
3. Coarse control layers $\rightarrow$ early layers govern high-lvl structural styles (eg: pose / face shape / etc).
4. Fine Layer Control $\rightarrow$ later layers control micro-lvl details (eg: hair texture / skin pigmentation / etc w/o altering facial geometry)

	DGGAN	CGAN	CycleGAN	StyleGAN
Key Idea	Uses deep strided/transposed convolutions.	Injects class labels into G & D.	Uses dual networks w/ cycle-consistency loss.	Injects disentangled styles via AdaIN layers.
Primary Application	Unconditional img generation	Targeted generation on demand.	Unpaired img-to-img translation	Controllable, high-res face synthesis

* Recs:

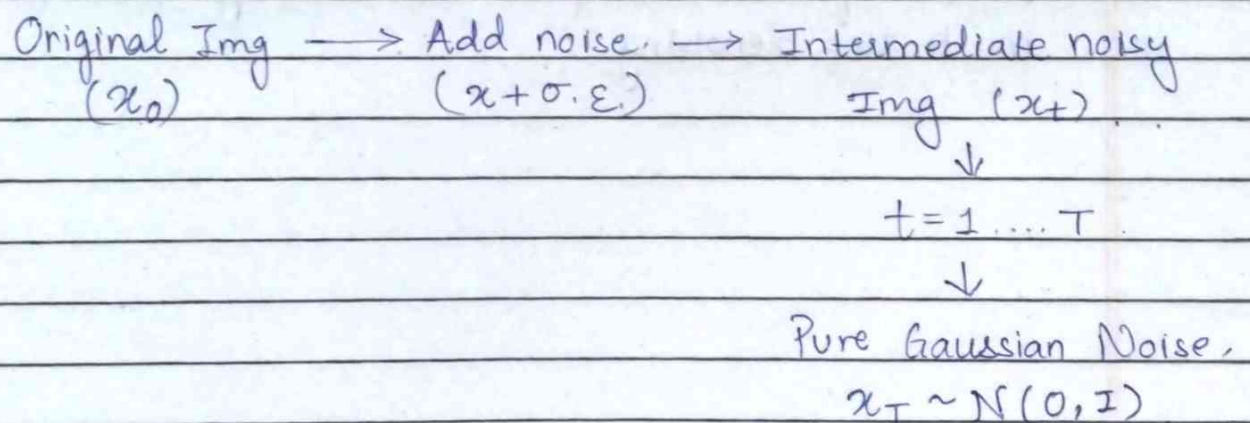
(i) Scenario: Horse to Zebra (unpaired) transformation,
↳ CycleGAN.

(ii) Scenario: Controllable High-Quality Face Synthesis
↳ StyleGAN.

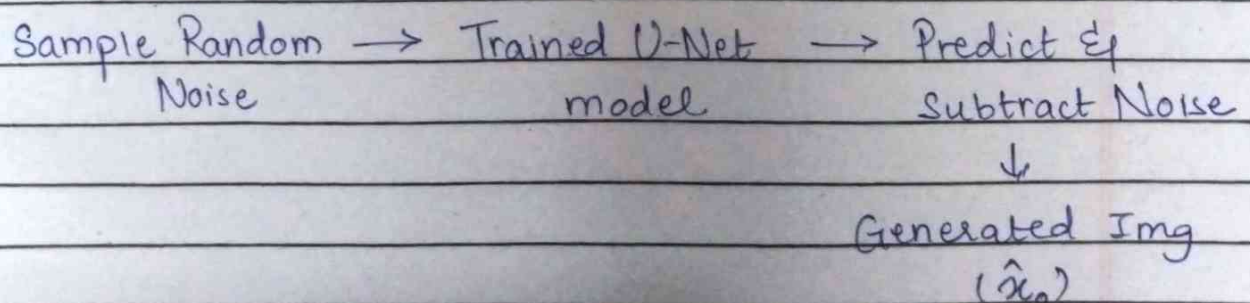
Q5 Intro. to Diffusion Models.

Diffusion models work by destroying img structures through gradual noise addition and training a neural network to reverse that exact degradation process.

1.* Forward Process (Noise Addition)



2.* Reverse Process (Learned Denoising)



* Choose model for:

23BT04175.

- (i) Realistic Face Generation $\rightarrow$ Diffusion model.
Generates high-fidelity photorealistic details w/o $\leftarrow$
suffering from mode collapse or training instability
like GANs.
- (ii) Molecule Design $\rightarrow$ VAE $\downarrow$
Learn smooth, continuous 2D/3D latent manifold; enables
continuous mathematical interpolation b/w valid
molecular structures.
- (iii) Img. Style Transfer $\rightarrow$ GANs.
Translates img across domains (eg: CycleGAN) in a
fast, single forward-pass w/o needing iterative
multi-step denoising.